	AIR UNIVERSITY
	Department Of Computer Engineering
	Experiment No : 3

Lab Tittle : Implement of Linked Lists in C++

Name: Muhammad Waqas **Reg No:240756**

Objectives: Implement a minimal singly linked list in C++ by defining a **Node**, explaining its non-contiguous memory arrangement, and tracing pointer updates for insertion.

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks : Obtained Marks :

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: Obtained Marks :

Date: 24th October , 24 Signature : Maryam



Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 3

Tittle : Link List

Name : Muhammad Waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 5h October 25

Lab Tasks :

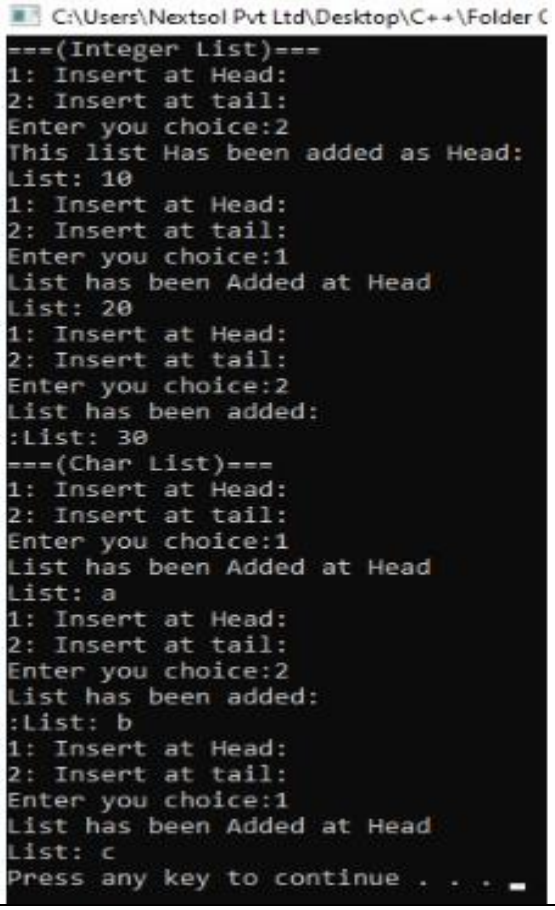
1) Make a function with name add_link. Whenever you call this function in the main, it will create a new node containing data.

CODE:

```
#include<iostream>
using namespace std;
template<typename T>
class node{
public:
T data;
node* next;
node(T val){
data=val;
next=NULL;} };
template<typename T>
class link_list{
private:
node<T>* head;
public:
link_list(){
head=NULL;}
void add_list(int val); };
template<typename T>
void link_list<T>::add_list(int val){
int choice;
node<T>* newnode=new node<T>(val);
cout<<"1: Insert at Head:\n";
cout<<"2: Insert at tail:\n";
cout<<"Enter you choice:";cin>>choice;
switch(choice){
case 1:
newnode->next=head;
head=newnode;
cout<<"List has been Added at Head\n"<<
"List: "<<newnode->data<<endl;
break;
```

```
case 2:
if(head==NULL){
head=newnode;
cout<<"This list Has been added as Head:\n"
<<"List: "<<newnode->data<<endl;}
else{node<T>* temp=head;
while(temp->next!=NULL){
temp=temp->next;}
temp->next=newnode;
cout<<"List has been added:\n"<<
":List: "<<newnode->data<<endl; }
        break;
        default:
            cout<<"This choice doesn't exist:\n";} }
int main(){
cout<<"===(Integer List)===\n";
link_list<int> l1;
l1.add_list(10);
l1.add_list(20);
l1.add_list(30);
//Char List
cout<<"===(Char List)===\n";
link_list<char> l2;
l2.add_list('a');
l2.add_list('b');
l2.add_list('c');
system("pause");}
```

OUTPUT:


C:\Users\Nextsol Pvt Ltd\Desktop\C++\Folder (
===(Integer List)===
1: Insert at Head:
2: Insert at tail:
Enter you choice:2
This list Has been added as Head:
List: 10
1: Insert at Head:
2: Insert at tail:
Enter you choice:1
List has been Added at Head
List: 20
1: Insert at Head:
2: Insert at tail:
Enter you choice:2
List has been added:
:List: 30
===(Char List)===
1: Insert at Head:
2: Insert at tail:
Enter you choice:1
List has been Added at Head
List: a
1: Insert at Head:
2: Insert at tail:
Enter you choice:2
List has been added:
:List: b
1: Insert at Head:
2: Insert at tail:
Enter you choice:1
List has been Added at Head
List: c
Press any key to continue . . .

2) Make a function which deletes node from your linked list at the tail and write a function to display all the data of your new linked list sequentially.

CODE:

```
> asd.cpp
using namespace std;
template<typename T>
class node{
public:
T data;
node* next;
node(T val){
data=val;
next=NULL;}};
template<typename T>
class link_list{
private:
node<T>* head;
public:
link_list(){
head=NULL;}
void add_list(int val){
node<T>* newnode=new node<T>(val);
if(head==NULL){
head=newnode;}
else{
node<T>* temp=head;
while(temp->next!=NULL){
temp=temp->next;}
temp->next=newnode;
cout<<"List has been added:"<<": "
<<temp->data<<endl; }}
void display_list(){
if(head==NULL){
cout<<"List is empty:\n";}
else{
```

```
else{
node<T>* temp=head;
cout<<"===(Link_List)===\n";
while(temp->next!=NULL){
cout<<"|"<<temp->data<<"|";
temp=temp->next;}
cout<<"NULL;";}}};
int main(){
cout<<"===(Integer List)===\n";
link_list<int> l1;
l1.add_list(10);
l1.add_list(20);
l1.add_list(30);
l1.display_list();
cout<<"===(Char List)===\n";
link_list<char> l2;
l2.add_list('a');
l2.add_list('b');
l2.add_list('c');
l2.display_list();
system("pause");}
```

SOLUTION:

```
C:\Users\Nextsol Pvt Ltd\Desktop\C++\Folder CPP\.vscode\a
===(Integer List)===
List has been added:: 10
List has been added:: 20
===(Link_List)===
|10||20|NULL;===(Char List)===
List has been added:: a
List has been added:: b
===(Link_List)===
|a||b|NULL;Press any key to continue . . .
```

3) Write and test a function to count the number of nodes in the linked list.

CODE:

```
e > asd.cpp
#include<iostream>
using namespace std;
template<typename T>
class node{
public:
T data;
node* next;
node(T val){data=val;
next=NULL;}};
template<typename T>
class link_list{
private:
node<T>* head;
public:
link_list(){head=NULL;}
void add_list(int val){
node<T>* newnode=new node<T>(val);
if(head==NULL){
head=newnode;}
else{node<T>* temp=head;
while(temp->next!=NULL){
temp=temp->next;}
temp->next=newnode;
cout<<"List has been added:"<<": "
<<temp->data<<endl; }          }
void display_list(){
if(head==NULL){
cout<<"List is empty:\n";}
else{
node<T>* temp=head;
cout<<"---(Link_List)---\n";
while(temp->next!=NULL){
```

```
while(temp->next!=NULL){
cout<<"|"<<temp->data<<"|";
temp=temp->next;}
cout<<"NULL;";}}
void display_addresslist(){
if(head==NULL){
cout<<"List is empty:\n";}
else{ node<T>* temp=head;
cout<<"---(Link_List)---\n";
while(temp->next!=NULL){
cout<<"|"<<temp<<"|";
temp=temp->next;}
cout<<"NULL;";}}
int countlist(){int count=0;
node<T>* temp=head;
while(temp->next!=NULL){count++;
temp=temp->next;}return count;}};
int main(){
link_list<int> l1;
l1.add_list(10);l1.add_list(20);
l1.add_list(30);l1.display_list();
cout<<"\nTotal Integer link lists:"
<<l1.countlist()<<endl;
link_list<char> l2;
l2.add_list('a');l2.add_list('b');
l2.add_list('c');l2.display_list();
cout<<"\nTotal Char link lists:"
<<l2.countlist()<<endl;
system("pause");}
```

OUTPUT:

```
C:\Users\Nextsol Pvt Ltd\Desktop\C++\Fol
List has been added:: 10
List has been added:: 20
===(Link_List)===
|10||20|NULL;
Total Integer link lists:2
List has been added:: a
List has been added:: b
===(Link_List)===
|a||b|NULL;
Total Char link lists:2
Press any key to continue . . .
```

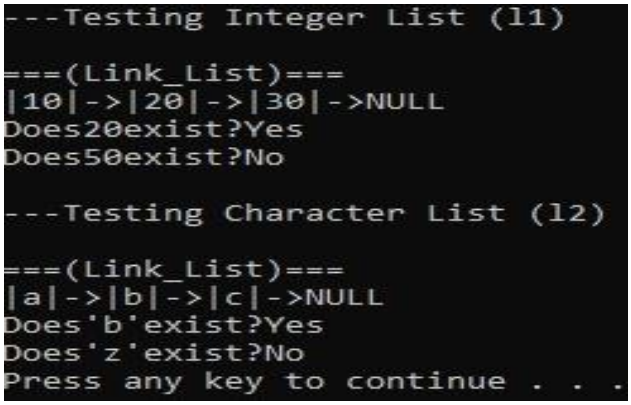
4) Write and function to find out whether the given data exists or not in the linked list.

CODE:

```
> asd.cpp
#include<iostream>
#include<cstdlib>
using namespace std;
template<typename T>
class node{
public: T data;
node<T>* next;
node(T val){data=val;next=NULL;}};
template<typename T>
class link_list{
private:
node<T>* head;
public:
link_list(){head=NULL;}
void add_list(T val){
node<T>* newnode=new node<T>(val);
if(head==NULL){head=newnode;}
else{
node<T>* temp=head;
while(temp->next!=NULL){temp=temp->next;}
temp->next=newnode;}}
void display_list(){
if(head==NULL){cout<<"List is empty.\n";}
else{
node<T>* temp=head;
cout<<"\n===(Link List)===\n";
while(temp!=NULL){
cout<<"|"<<temp->data<<"|->";
temp=temp->next;}
cout<<"NULL\n";}}
bool exists(T val){
node<T>* temp=head;
```

```
class link_list{
bool exists(T val){
while(temp!=NULL){
if(temp->data==val){return true;}
temp=temp->next;}
return false;}};
int main(){
link_list<int> l1;
cout<<"---Testing Integer List (l1)\n";
l1.add_list(10);
l1.add_list(20);
l1.add_list(30);
l1.display_list();
int search_val_1=20;
cout<<"Does"<<search_val_1<<"exist?"<<
(l1.exists(search_val_1)?"Yes":"No")<<endl;
int search_val_2=50;
cout<<"Does"<<search_val_2<<"exist?"
<<(l1.exists(search_val_2)?"Yes":"No")<<endl;
link_list<char> l2;
cout<<"\n---Testing Character List (l2)\n";
l2.add_list('a');
l2.add_list('b');
l2.add_list('c');
l2.display_list();
char search_char_1='b';
cout<<"Does'"<<search_char_1<<"'exist?"
<<(l2.exists(search_char_1)?"Yes":"No")<<endl;
char search_char_2='z';
cout<<"Does'"<<search_char_2<<"'exist?"
<<(l2.exists(search_char_2)?"Yes":"No")<<endl;
system("pause");}
```

OUTPUT:


---Testing Integer List (l1)
===(Link_List)===
|10|->|20|->|30|->NULL
Does20exist?Yes
Does50exist?No
---Testing Character List (l2)
===(Link_List)===
|a|->|b|->|c|->NULL
Does'b'exist?Yes
Does'z'exist?No
Press any key to continue . . .

Conclusion :

- The primary advantage over arrays is that linked list nodes do not require **contiguous memory allocation**, allowing for flexible memory use and avoiding allocation issues.
- The **head pointer** stores the address of the first node, and the **link field** of the last node is set to **null**.
- The lab objectives focus on defining the **Node structure**, understanding the **non-contiguous** memory arrangement, and implementing a minimal **singly linked list** in C++.
- Students will learn to trace and diagram **pointer updates** (dry run) for insertion operations to understand how the logical order is maintained independently of physical memory.
- This structure is essential for scenarios requiring frequent insertions or deletions without the overhead of shifting elements.